

SoftwareProject



**UNIVERSITY of INFORMATION
TECHNOLOGY and MANAGEMENT**
in Rzeszow, POLAND

JUDY KATHRYN W71045

VIMBAI DANDURI W71128

MOUIN KHAZRI W69521

SOFTWARE PROJECT

FoodieNetwork

Cooking Lovers Application

Chosen Technology Stack.....	3
Backend Technologies:-.....	3
-AppWrite.....	3
Frontend Technologies:-.....	4
-React.....	4
-TanStack Query.....	4
-Tailwind CSS.....	5
Application Architecure.....	7
Frontend:.....	7
Backend:.....	7
Data Flow:.....	7
Description of Application Functionality.....	8
User Interface Description.....	9
Summary.....	10
Sources:-.....	11

Chosen Technology Stack

For our Cooking Lovers Application, we decided to work mainly with React and Appwrite frameworks for the Backend. Along with HTML and CSS for the Frontend to help with making the platform look more appealing and user-friendly.

Backend Technologies:-

-AppWrite

Appwrite is a open-source backend-as-a-service that offers authentication, databases and storages.

We chose Appwrite because it allows you to not build a backend from scratch, it provides serverless functions, databases, authentication and storages as a package,

Key Features:

- **Authentication** – Supports email/password, OAuth (Google, GitHub, etc.), and anonymous login.
- **Database – NoSQL document-based database with real-time syncing.**
- **Storage** – File storage and management for images, videos, and documents.
- **Serverless Functions** – Run backend logic with cloud functions in multiple languages.
- **APIs & SDKs** – Easy integration with REST, GraphQL, and SDKs for various platforms.
- **Real-time Capabilities** – Get live updates on database changes without manual refreshing.
- **Self-Hosting Option** – Deploy on your own server for full control over data privacy.

Appwrite serves as the backend, managing user authentication, databases, storage, and serverless functions, while the frontend (React.js) handles the UI. It provides a REST and GraphQL API, allowing your app to interact with backend services securely. With real-time updates, Appwrite ensures that changes in data are instantly reflected on the client side without requiring manual refreshes. Since it supports self-hosting, you can have full control over your data, making it a flexible alternative to other backend solutions.

Frontend Technologies:-

-React

React(ReactJS/React.js) is a free and open-source front-end JavaScript library that aims to make building user interfaces based on components more "seamless"

Because React is only concerned with the user interface and rendering components to the DOM, React applications often rely on libraries for routing and other client-side functionality. A key advantage of React is that it only re-renders those parts of the page that have changed, avoiding unnecessary re-rendering of unchanged DOM elements.

Key Features:

- **Component-Based** – UI is built using reusable components.
- **Virtual DOM** – Efficient updates by comparing and modifying only changed parts.
- **Declarative UI** – Simplifies UI development by describing how the UI should look based on state.
- **State Management** – Uses `useState` and `useReducer` for managing component state.
- **Props** – Pass data between components easily.
- **Hooks** – Enable functional components to use state and lifecycle features (`useEffect`, `useContext`, etc.).
- **One-Way Data Binding** – Ensures predictable data flow.
- **Fast Rendering** – Uses reconciliation to optimize performance.
- **React Router** – Enables navigation in single-page applications (SPA).
- **Rich Ecosystem** – Works well with libraries like Redux, Next.js, and TailwindCSS.

React's component-based structure, efficient rendering, and state management make it the perfect choice for building FoodieNetwork, ensuring a seamless, interactive, and scalable user experience.

-TanStack Query

TanStack Query simplifies **data fetching, caching, and state synchronization** in React applications. It optimizes API interactions by handling background updates, caching responses, and reducing unnecessary network requests.

Key Features:

- **Automatic Caching** – Stores API responses to reduce redundant requests.
- **Background Updates** – Refreshes stale data without requiring a page reload.
- **Optimistic UI** – Updates UI instantly while awaiting API responses.
- **Pagination & Infinite Scrolling** – Manages large datasets efficiently.
- **Mutation Management** – Handles create, update, and delete API requests seamlessly.

HTML provides the **structure** of a webpage, defining content elements that browsers render visually. Combined with CSS for styling and JavaScript(React and Appwrite) for interactivity, it forms the foundation of web development.

-Tailwind CSS

Tailwind CSS is a utility-first CSS framework that provides pre-defined classes to style elements without writing traditional CSS. It enables rapid UI development by allowing direct styling in HTML or JSX. Unlike conventional CSS frameworks, it doesn't rely on prebuilt components but instead focuses on flexibility and customization.

Key Features:

- **Utility-First Approach** – Applies styles directly in HTML or JSX without writing separate CSS.
- **Responsive Design** – Uses built-in breakpoints for mobile-first development.
- **Customization** – Allows full control over styles through `tailwind.config.js`.
- **Dark Mode Support** – Enables theme switching with minimal configuration.
- **Performance Optimized** – Removes unused CSS using PurgeCSS for smaller file sizes.

Tailwind will be making our code look cleaner, organized, and more readable thanks to its' pre-defined classes which allows styling elements without the need for traditional CSS.

-ShadCN

ShadCN provides a collection of pre-styled, accessible UI components built on Radix UI and Tailwind CSS. It allows for quick integration of modern, themeable components while maintaining full customization

Key Features:

- **Pre-Styled Components** – Offers ready-to-use UI elements like buttons, inputs, and modals.
- **Accessibility Focused** – Built on Radix UI to ensure WCAG compliance.
- **Customizable Themes** – Supports design consistency with Tailwind CSS integration.
- **Dark Mode Support** – Automatically adapts to user preferences.
- **Flexible & Unopinionated** – Allows modifications without being locked into a design system.

Application Architecture

Application architecture is the process of defining a structure that meets all the technical and operational requirements of an application. It's like a blueprint that shows the design of a system and portrays how its' components work together to achieve an objective.

We have chosen **FoodieNetwork** as the name for our application.

Frontend:

Using React.js and Tailwind CSS for the User Interface to enable interactivity with the app, the rendering of the webpages such as login, sign up, profiles, the feed, and post recipes.

-Data like recipe details will be displayed, along with community engagement like comments and likes, as well as recipe sharing. This data is sent to the backend side of the application.

Backend:

Our choice of frameworks was selected to aid in building the backbone of the app, allowing us to process the user's requests(engagement, sharing, liking, commenting, etc) and manage the data.

-Anything related with making the application actually function like any other social media platform is handled in the backend.

Data Flow:

-The frontend is used for user interaction, while the backend receives the data through form submission or using HTTP requests, and then handles and processes it.

-The backend will once again transfer the data it processed back to the frontend, updating the User Interface(number of likes, new recipe posted, comments, etc),

-Content uploaded by the user will be store in the media folder and managed by Appwrite.

Description of Application Functionality

Our app will include a login page for pre-existing registered users, alternatively, new users can sign up and create an account.

On the user's feed, followed accounts and their posted content along with recipes will be hosted there. A popular dishes section will also be included in the homepage(feed).

The search bar included will allow users to search for almost anything within the app, from other users, to recipes, and generated grocery lists.

Users will be able to like, comment and share other's posts/recipes, or their own content if needed.

Saving a recipe for later is also an added feature.

Shopping lists can be generated based on the user's wanted recipe.

Account management will be included, deleting posts or account, managing other user's, along with profile pictures and security measures.

User Interface Description

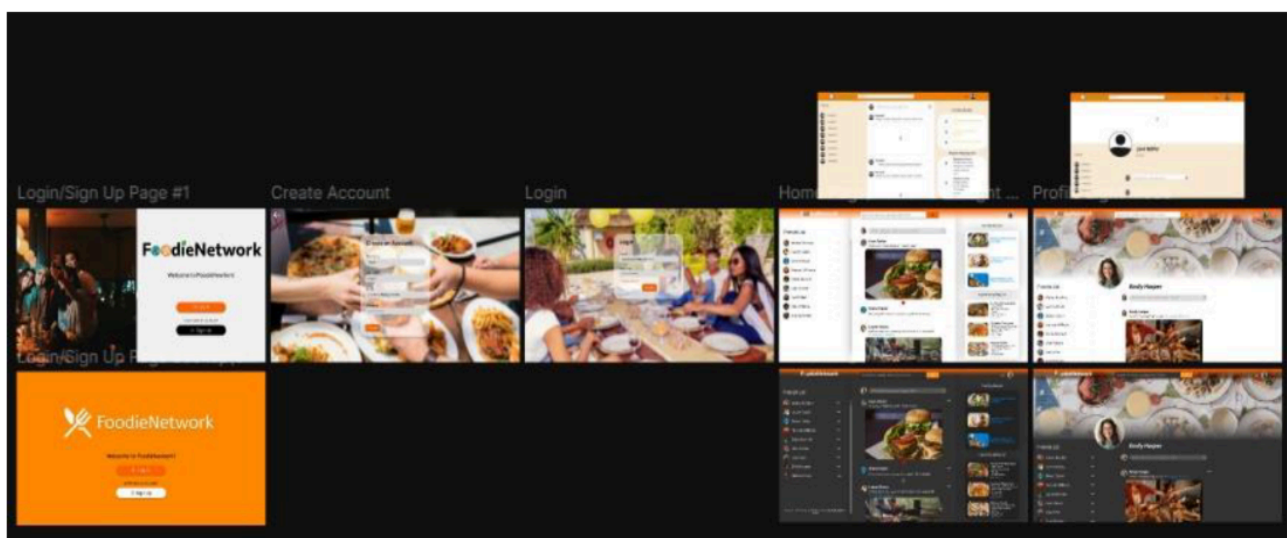
The chosen primary color for our application is orange, which is an energetic color commonly involved with freshness and cuisine, giving the user a feeling of warmth, excitement, and in the right place. A clean and modern design while maintaining noise and loud to keep the users intrigued.

An abundance of buttons and icons were provided to make the User Experience enjoyable and easy.

Pages Included:-

- Login - enabling users to access their accounts
- Sign Up - new users will also have the possibility of engaging with our app
- Homepage - everything will be found here, access to profile, friends, posts, trending recipes, and anything else the user might need
- Profile - account management, deleting account, removing followers, user's shared content, personal details

With time, we hope to make our app accessible on mobile phones.



Summary

Our app, as the name indicates, is for cooking lovers and enthusiasts hoping to connect and share their passion for cuisine with the world and individuals of similar interests.

HTML, CSS, are primarily used for the frontend due to ease of navigation and flexibility.

Figma was used to design what the app should look like in its final form.

React and Appwrite were the most suitable options for our backend side of the app because it fulfills our needs, being a versatile and powerful framework without adding complication to usage.

We hope to add more features in the future to make this app as functional as possible.

Sources:-

wikipedia, figma, w3schools.com, chatgpt